In [1]:
```python
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error
```

# Q1: Feature Scaling Concepts

**Answer:**

- **Standardization**: Centers the data around a mean of 0 and scales it to have a standard deviation of 1. It assumes the data follows a Gaussian distribution.
- **Min-Max Scaling**: Scales the data into a fixed range, typically between 0 and 1. It is sensitive to outliers.
- **Robust scaling**: Uses statistics that are robust to outliers, such as the median and the interquartile range (IQR), instead of mean and standard deviation.
- **Why feature scaling is important**: It ensures that all features contribute equally to the distance calculations and cost functions, speeding up gradient descent convergence and improving the performance of distance-based algorithms like KNN and SVM.

# Q2: Load and Explore Dataset

In [2]:
```python
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print(X.shape)
display(X.head())
```

# Q3: Train-Test Split

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

## Q4: Apply Feature Scaling

In [4]:
```
std_scaler = StandardScaler()
X_train_std = std_scaler.fit_transform(X_train)
X_test_std = std_scaler.transform(X_test)

mm_scaler = MinMaxScaler()
X_train_mm = mm_scaler.fit_transform(X_train)
X_test_mm = mm_scaler.transform(X_test)

rob_scaler = RobustScaler()
X_train_rob = rob_scaler.fit_transform(X_train)
X_test_rob = rob_scaler.transform(X_test)
```

## Q5: Train Models

In [5]:
```
# Train unscaled data
model_unscaled = LogisticRegression(max_iter=10000)
model_unscaled.fit(X_train, y_train)
```

In [6]:
```
#Train standardized scaled data
model_std = LogisticRegression()
model_std.fit(X_train_std, y_train)
```

In [7]:
```
# Train MinMax scaled data
model_mm = LogisticRegression()
model_mm.fit(X_train_mm, y_train)
```

In [8]:
```
#Train Robust scaled data
model_rob = LogisticRegression()
model_rob.fit(X_train_rob, y_train)
```

# Q6: Evaluate Each Model

In [9]:
```
def eval_metrics(y_true, y_pred):
    return accuracy_score(y_true, y_pred), mean_absolute_error(y_true, y_pred), np.sqrt(mean_squared_error(y_true, y_pred))
```

In [10]:
```
# For unscaled data
pred_unscaled = model_unscaled.predict(X_test)
acc_u, mae_u, rmse_u = eval_metrics(y_test, pred_unscaled)
print(f"Unscaled - Acc: {acc_u:.4f}, MAE: {mae_u:.4f}, RMSE: {rmse_u:.4f}")
```

In [11]:
```
# For standardized scaled data
pred_std = model_std.predict(X_test_std)
acc_s, mae_s, rmse_s = eval_metrics(y_test, pred_std)
print(f"Standardized - Acc: {acc_s:.4f}, MAE: {mae_s:.4f}, RMSE: {rmse_s:.4f}")
```

In [12]:
```
# For MinMax scaled data
pred_mm = model_mm.predict(X_test_mm)
acc_m, mae_m, rmse_m = eval_metrics(y_test, pred_mm)
print(f"MinMax - Acc: {acc_m:.4f}, MAE: {mae_m:.4f}, RMSE: {rmse_m:.4f}")
```

In [13]:
```
# For Robust scaled data
pred_rob = model_rob.predict(X_test_rob)
acc_r, mae_r, rmse_r = eval_metrics(y_test, pred_rob)
print(f"Robust - Acc: {acc_r:.4f}, MAE: {mae_r:.4f}, RMSE: {rmse_r:.4f}")
```

| Scaler type | Accuracy | MAE | RMSE |
|---|---|---|---|
| No Scaling | 0.9561 | 0.0439 | 0.2094 |
| STD scaling | 0.9737 | 0.0263 | 0.1622 |
| MinMax Scaling | 0.9825 | 0.0175 | 0.1325 |
| Robust Scaling | 0.9825 | 0.0175 | 0.1325 |

# Q8: Reflection

**Answer:**

- **Impact of scaling on performance**: Scaling clearly improved the performance of the Logistic Regression model, boosting accuracy and lowering MAE/RMSE. Without scaling, the model struggled to penalize features efficiently and required many more iterations to converge.
- **When scaling is necessary**: Scaling is essential when algorithms use distance-based metrics (like KNN, SVM, K-Means) or gradient descent for optimization (like Logistic Regression, Neural Networks). It's also necessary when features in the dataset have vastly different ranges or units.